1. Présentation générale

Agentic AI Copilot est un assistant intelligent destiné aux étudiants, développeurs et ingénieurs souhaitant apprendre et exploiter les technologies de l'Agentic AI pour concevoir et développer leurs propres systèmes intelligents autonomes.

Aujourd'hui, le développement d'applications basées sur l'Agentic AI nécessite la maîtrise de plusieurs concepts : agents intelligents, LLMs, tool calling, mémoire, planification, workflows, orchestration multi-agents, RAG, MCP et utilisation d'outils externes.

Ces notions sont documentées dans de nombreuses ressources et frameworks, tels que LangGraph, CrewAI, AutoGen et les outils de développement d'agents. Il est parfois difficile de comprendre leurs différences, de choisir les bonnes technologies et de savoir comment les intégrer dans un projet concret.

Notre solution consiste à développer un assistant intelligent capable d'accompagner l'utilisateur tout au long du processus de création d'un système Agentic AI : de la compréhension de son idée jusqu'à la conception de son architecture et l'implémentation technique.

Le système combinera **Agentic AI et Hybrid GraphRAG** afin de fournir des réponses contextualisées, d'explorer les relations entre les concepts et de proposer des recommandations adaptées aux besoins de chaque projet.

2. Problématique

L'écosystème de l'Agentic AI évolue rapidement et comporte de nombreux frameworks, architectures, outils et concepts interdépendants. Les développeurs rencontrent des difficultés pour comprendre leur fonctionnement, choisir les technologies adaptées à leurs besoins et concevoir correctement leurs propres systèmes multi-agents.

Comment concevoir un assistant intelligent capable d'accompagner les développeurs dans l'apprentissage, la conception et l'implémentation de systèmes Agentic AI, en exploitant les documentations techniques et les relations entre les différents concepts, frameworks et outils ?

3. Objectifs du projet

L'objectif principal est de proposer un véritable copilote de développement spécialisé dans l'Agentic AI.

Il devra permettre à l'utilisateur de :

- Comprendre les concepts fondamentaux et avancés de l'Agentic AI.
- Découvrir et comparer les frameworks de développement d'agents.
- Choisir une architecture adaptée à son besoin : agent unique, multi-agents ou workflow hybride.
- Identifier les outils nécessaires à la réalisation de son projet.
- Concevoir l'architecture d'un système Agentic AI.
- Obtenir des exemples de code et des explications d'implémentation.
- Comprendre les mécanismes de mémoire, de planification, de raisonnement et de tool calling.
- Diagnostiquer et corriger des problèmes dans ses workflows agentiques.
- Découvrir les bonnes pratiques d'évaluation, de sécurité et de fiabilité des agents.

4. Fonctionnalités principales

A. Agentic AI Learning Assistant

Un assistant pédagogique qui explique les concepts de l'Agentic AI avec des exemples pratiques.

Exemples :

- Quelle est la différence entre un LLM classique et un AI Agent ?
- Comment fonctionne le tool calling ?
- Quelle est la différence entre un workflow et un agent autonome ?
- Comment fonctionne la mémoire d'un agent ?
- Quand utiliser une architecture multi-agents ?

B. Framework Recommendation

Un module qui aide l'utilisateur à choisir les frameworks et technologies adaptés à son besoin.

Exemple :

« Je souhaite développer un système multi-agents dans lequel plusieurs agents collaborent pour effectuer une recherche documentaire, analyser les résultats et générer un rapport. Quels frameworks et quelles architectures me recommandes-tu ? »

L'assistant pourra comparer les frameworks, expliquer leurs avantages et proposer une solution adaptée.

C. Agentic Architecture Designer

Un module qui transforme une idée de projet en architecture technique.

L'utilisateur décrit son besoin et l'assistant propose :

- Le type d'architecture.
- Le nombre et les responsabilités des agents.
- Les outils nécessaires.
- Les interactions entre les agents.
- La gestion de la mémoire et de l'état.
- Le workflow d'exécution.
- Les technologies recommandées.

D. Code Assistant

Un assistant qui accompagne l'utilisateur dans l'implémentation de ses systèmes.

Il pourra :

- Générer des exemples de code.
- Expliquer le fonctionnement des composants.
- Proposer une structure de projet.
- Aider à intégrer des outils externes.
- Expliquer les erreurs rencontrées.
- Proposer des corrections et des améliorations.

E. Agentic Workflow Explorer

Un module qui permet de comprendre et de visualiser le fonctionnement des architectures agentiques.

Il pourra présenter les interactions entre les agents, leurs responsabilités, les outils utilisés, les étapes d'exécution et les conditions de transition.

F. Evaluation and Optimization Assistant

Un module qui aide à évaluer et améliorer les systèmes créés.

Il pourra expliquer :

- Comment mesurer la qualité des réponses.
- Comment évaluer l'utilisation des outils.
- Comment détecter les erreurs d'exécution.
- Comment limiter les hallucinations.
- Comment mesurer la fiabilité et les performances d'un système multi-agents.

5. Utilisation du Hybrid GraphRAG

Le GraphRAG sera utilisé pour représenter les relations entre les concepts, technologies, frameworks et composants de l'Agentic AI.

Nous construirons un graphe de connaissances à partir de documentations officielles et de ressources techniques fiables.

Exemples d'entités :

- Concepts : AI Agent, Multi-Agent System, Planning, Memory, Tool Calling, RAG.
- Frameworks : LangGraph, CrewAI, AutoGen.
- Composants : Agent, Tool, State, Node, Workflow, LLM.
- Protocoles et technologies : MCP, API, Vector Database.
- Techniques : Reflection, Planning, Routing, Retrieval.

Exemples de relations :

- LangGraph IMPLEMENTS agent workflows.
- Agent USES Tool.
- Agent HAS Memory.
- Agent DELEGATES_TO another Agent.
- Workflow CONTAINS Nodes.
- Agent USES LLM.
- RAG USES Vector Database.
- MCP CONNECTS Agents to external tools.

Ces relations seront extraites des sources documentaires, vérifiées et stockées dans Neo4j avec leurs références.

Le système disposera également d'une base vectorielle FAISS permettant de rechercher les passages les plus pertinents.

La combinaison des deux permettra de répondre à des questions simples, mais également d'explorer des relations complexes entre les différents composants de l'Agentic AI.

6. Architecture technique envisagée

L'application reposera sur les composants suivants :

Composant	Technologies
Langage	Python
Orchestration agentique	LangGraph
LLM	Groq API ou Ollama
Embeddings	Sentence Transformers
Recherche vectorielle	FAISS

Graphe de connaissances	Neo4j
Backend	FastAPI
Frontend	React
Visualisation	Bibliothèque de graphes interactive

L'assistant pourra utiliser une architecture agentique comprenant plusieurs rôles :

- **Intent Analyzer :** comprend le besoin de l'utilisateur.
- **Knowledge Retrieval :** consulte les documentations via la recherche vectorielle et le graphe.
- **Architecture Advisor :** propose une architecture et les technologies adaptées.
- **Code Assistant :** fournit des exemples et accompagne l'implémentation.
- **Evaluation Assistant :** vérifie les recommandations et aide à évaluer les solutions proposées.

Un orchestrateur contrôlera les interactions et déterminera les étapes nécessaires selon la demande.

7. Exemple concret d'utilisation

Un développeur souhaite créer un assistant intelligent capable de rechercher des informations sur Internet, d'analyser les résultats et de produire automatiquement un rapport.

Il pose la question :

« Je veux développer un système Agentic AI avec plusieurs agents qui collaborent pour effectuer une recherche sur Internet, analyser les informations et générer un rapport. Comment dois-je concevoir mon architecture ? »

L'assistant va :

1. Analyser le besoin et identifier les fonctionnalités demandées.
2. Identifier les concepts concernés : multi-agent orchestration, web search, planning, tools et report generation.
3. Consulter les documentations techniques pertinentes.
4. Explorer les relations entre les frameworks et les composants dans le graphe de connaissances.
5. Comparer les architectures et les technologies adaptées.
6. Proposer une architecture avec les responsabilités de chaque agent.
7. Fournir une structure de projet et des exemples de code.
8. Présenter les recommandations avec les références documentaires.

Le développeur peut ensuite poser des questions complémentaires pour approfondir ou modifier l'architecture proposée.

8. Corpus documentaire

Le corpus sera principalement constitué de documentations officielles relatives à l'Agentic AI :

- Documentation LangGraph.
- Documentation CrewAI.
- Documentation AutoGen.
- Documentation des outils de développement d'agents.
- Documentation MCP.
- Ressources sur le function calling et l'utilisation d'outils.
- Ressources sur les architectures RAG et GraphRAG.
- Documentation sur l'évaluation et la fiabilité des agents.

Le corpus devra être organisé par framework, concept, composant et version afin de préserver la pertinence et l'actualité des réponses.

9. Évaluation

Nous évaluerons notre assistant à travers plusieurs types de questions :

- Questions conceptuelles sur l'Agentic AI.
- Questions de comparaison entre frameworks.
- Questions de conception d'architecture.
- Questions portant s